PYSPARK COMPLETE NOTES

PHASE 4

Advanced Transformations

Aggregate, pivot, join - then understand shuffles, tuning and data-quality checks.

YOU WILL

- ✓ Group by one or many columns and compute sum, avg and more
- ✓ Gather values into lists with collect_list and reshape with pivot
- ✓ Build if / else-if / else columns with when and otherwise
- ✓ Run inner, left, right and anti joins and predict their rows
- ✓ Explain shuffles, repartition vs coalesce, cache vs persist and broadcast joins
- ✓ Write reusable data-quality checks and find unmatched join keys

START HERE IF

Phase 3 (functions, null handling, arrays).

Built from a Databricks PySpark tutorial notebook and its course outline.
Clickable contents, links and bookmarks work in any PDF viewer.

Phase 4 · Advanced Transformations

Aggregate, pivot, join - then understand shuffles, tuning and data-quality checks.

YOU WILL BE ABLE TO

- ✓ Group by one or many columns and compute sum, avg and more
- ✓ Gather values into lists with collect_list and reshape with pivot
- ✓ Build if / else-if / else columns with when and otherwise
- ✓ Run inner, left, right and anti joins and predict their rows
- ✓ Explain shuffles, repartition vs coalesce, cache vs persist and broadcast joins
- ✓ Write reusable data-quality checks and find unmatched join keys

START HERE IF

Phase 3 (functions, null handling, arrays).

IN THIS PDF

Explanations, diagrams and code from the notebook · sections tagged **Beyond the notebook** · a decoder for every term · interview questions · a data-engineering lens · a self-test with answers · a glossary

How to read the boxes

IN PLAIN WORDS
A jargon-free explanation of the idea.

DECODE THE CODE
Meaning of every function, option and symbol in the code above (first time it appears).

TIP
A habit that makes your code cleaner, faster or safer.

WATCH OUT
A trap that gives a wrong or surprising result.

NOTEBOOK OBSERVATION
Something specific noticed in the course notebook.

DATA ENGINEERING LENS
How the skill is used in real pipelines (summary page at the end).

EXTRA
Extra knowledge that is not in the notebook.

BEYOND THE NOTEBOOK
A tag under a heading: new material added to the notebook's topics, ending with an Interview corner.

CONTENTS (click an entry to jump to it)

4.1 groupBy() and agg() . . . 3
4.2 collect_list() . . . 5
4.3 pivot() . . . 6
4.4 when() and otherwise() . . . 7
4.5 Joins . . . 9
4.6 Shuffles and partitions . . . 12
4.7 repartition() vs coalesce() . . . 13
4.8 cache() vs persist() . . . 15
4.9 Broadcast joins and join strategies . . . 16
4.10 A basic tuning checklist . . . 17
4.11 Data-quality checks . . . 18
4.12 Reusable checks, fail-fast vs recording failures . . . 22
From notebook to pipeline . . . 26
Glossary . . . 26
Check yourself . . . 30
Answers and solutions . . . 31

4.1 groupBy() and agg()

IN PLAIN WORDS

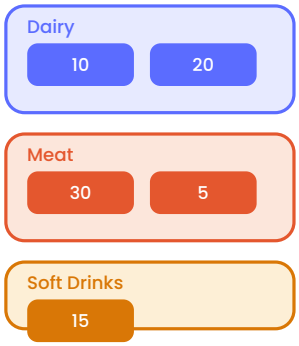
Sort a pile of receipts into one stack per shop, then total each stack. **groupBy** makes the stacks; **agg** (aggregate) calculates one number per stack. Many rows go in, one row per group comes out.

The pattern is `df.groupBy(columns).agg(functions)`. Common aggregate functions:

MANY ROWS IN

Item_Type	Item_MRP
Dairy	10
Meat	30
Dairy	20
Soft Drinks	15
Meat	5

GROUPBY('ITEM_TYPE') MAKES STACKS



.agg()

ONE ROW PER GROUP OUT

Item_Type	sum(Item_MRP)
Dairy	30
Meat	35
Soft Drinks	15

groupBy(columns).agg(functions) - many rows go in, ONE row per group comes out.
Group by two columns (Item_Type + Outlet_Size) -> one row per combination. Illustrative numbers.

Function	Result per group
`sum('c')`	Total
`avg('c')`	Average
`min('c') / max('c')`	Smallest / largest
`count('c')`	Number of non-null values

Scenario 1 - total of one column per group

PYSPARK

```
df.display()
```

DECODE THE CODE

`.display()`	Databricks' table viewer. It is an **action**, so it makes Spark run the job.

PYSPARK

```
df.groupBy('Item_Type').agg(sum('Item_MRP')).display()
```

DECODE THE CODE

`.groupBy(columns)`	Collects rows that share the same key(s) into groups. Needs an aggregate next.
`.agg(...)`	Calculates one or more aggregates for every group.
`sum(col)`	Total of a column (Spark's own *sum*, available through the import *).

SHAPE OF THE RESULT: ONE ROW PER Item_Type

Item_Type	sum(Item_MRP)
Dairy	total MRP of all Dairy rows
Soft Drinks	total MRP of all Soft Drinks rows
Meat	...

Spark names the new column `sum(Item_MRP)` automatically. This works because `sum` is Spark's function (see the import warning in Phase 1).

Scenario 2 - average

PYSPARK

```
df.groupBy('Item_Type').agg(avg('Item_MRP')).display()
```

DECODE THE CODE

`avg(col)`	Average of a column.

Scenario 3 - group by two columns, and name the result

PYSPARK

```
df.groupBy('Item_Type','Outlet_Size').agg(sum('Item_MRP').alias('Total_MRP')).display()
```

DECODE THE CODE

`.alias('new')`	Gives a column a new name **in this result only**.

Grouping by two columns gives **one row per combination** (for example Dairy + Medium, Dairy + Small ...). `.alias('Total_MRP')` replaces the clumsy automatic name. Rows whose Outlet_Size is null form a group of their own.

Scenario 4 - several aggregates at once

PYSPARK

```
df.groupBy('Item_Type','Outlet_Size').agg(sum('Item_MRP'),avg('Item_MRP')).display()
```

SHAPE OF THE RESULT

Item_Type	Outlet_Size	sum(Item_MRP)	avg(Item_MRP)
Dairy	Medium	...	...
Dairy	Small	...	...

NAME YOUR AGGREGATES

Write `sum('Item_MRP').alias('Total_MRP')` every time. Names like `sum(Item_MRP)` contain brackets, are awkward to refer to later, and break when you save the result as a table.

4.2 collect_list()

`collect_list` is an aggregate that gathers all the values of a group into one **array**. Think of it as "put everything of this group in one cell".

DF_BOOK

user	book
user1	book1
user1	book2
user2	book2
user2	book4
user3	book1

groupBy('user')
.agg(collect_list('book'))

ONE ROW PER USER, VALUES GATHERED INTO AN ARRAY

user	collect_list(book)
user1	[book1, book2]
user2	[book2, book4]
user3	[book1]

explode() turns an array back into one row per item

collect_list is an aggregate: it must follow groupBy. The order inside each list is not guaranteed.

PYSPARK

```
data = [('user1','book1'),
        ('user1','book2'),
        ('user2','book2'),
        ('user2','book4'),
        ('user3','book1')]

schema = 'user string, book string'

df_book = spark.createDataFrame(data,schema)

df_book.display()
```

DECODE THE CODE

`[('1','kad'), ...]`	A Python list of tuples. Each tuple is one row of the DataFrame.
`spark`	The SparkSession - your entry point to Spark. Databricks creates it for you, already named **spark**.
`spark.createDataFrame(rows, schema)`	Builds a DataFrame from Python data.
`spark.createDataFrame(data, schema)`	Builds a small DataFrame from a Python list plus a schema string.

df_book

user	book
user1	book1
user1	book2
user2	book2
user2	book4
user3	book1

PYSPARK

```
df_book.groupBy('user').agg(collect_list('book')).display()
```

DECODE THE CODE

`collect_list(col)`	Gathers a group's values into one **array** (the opposite of explode).

RESULT: ONE ROW PER USER

user	collect_list(book)
user1	[book1, book2]
user2	[book2, book4]
user3	[book1]

The order of the items inside each list is not guaranteed. The notebook then selects three columns, `Item_Type`, `Outlet_Size` and `Item_MRP`, as the input for the pivot that follows:

PYSPARK

```
df.select('Item_Type','Outlet_Size','Item_MRP').display()
```

DECODE THE CODE

`.select(...)`	Keeps only the listed columns (or new calculated columns) - like SQL SELECT.

COLLECT_SET

PYSPARK · EXTRA

```
df_book.groupBy('user').agg(collect_set('book'))   # like collect_list but without repeated values
```

DECODE THE CODE

`collect_set(col)`	Like collect_list but without repeated values.

4.3 pivot()

Pivot turns the *distinct values of one column into column headings*. Compare a long list of (Item_Type, Outlet_Size, price) rows with a compact grid: one row per Item_Type and one column per Outlet_Size.

LONG FORM (ROWS)

Item_Type	Outlet_Size	Item_MRP
Dairy	Small	10
Dairy	Medium	20
Meat	Small	30
Meat	Medium	50

.pivot()

WIDE FORM (GRID)

Item_Type	Small	Medium
Dairy	10	20
Meat	30	50

rows = groupBy

headings = distinct values of Outlet_Size

cells = agg(avg('Item_MRP'))

```
df.groupBy('Item_Type').pivot('Outlet_Size').agg(avg('Item_MRP'))
```

groupBy -> becomes the ROWS
pivot -> its distinct values become the COLUMN HEADINGS
agg -> fills each cell (null Outlet_Size values get a column titled 'null')

PYSPARK

```
df.groupBy('Item_Type').pivot('Outlet_Size').agg(avg('Item_MRP')).display()
```

DECODE THE CODE

`.pivot(col)`	Turns the distinct values of a column into **column headings**.

Part	Role
`groupBy('Item_Type')`	Becomes the **rows** of the grid.
`pivot('Outlet_Size')`	Its distinct values become the **column headings**.
`agg(avg('Item_MRP'))`	Fills each cell: the average MRP for that Item_Type in that Outlet_Size.

SHAPE OF THE RESULT (EACH 'avg' IS A NUMBER)

Item_Type	null	High	Medium	Small
Dairy	avg	avg	avg	avg
Meat	avg	avg	avg	avg

WHERE DOES THE NULL COLUMN COME FROM?

Outlet_Size has missing values, so Spark creates a column for them too, titled **null**. Fill the nulls first with `fillna` if you want a tidy heading.

FASTER PIVOTS

Spark first has to find the distinct values of the pivot column. If you already know them, pass them in: `pivot('Outlet_Size', ['High','Medium','Small'])`.

4.4 when() and otherwise()

when / otherwise is Spark's if / else-if / else, used to create a column whose value depends on conditions - the equivalent of SQL's `CASE WHEN`.

Scenario 1 - one condition

PYSPARK

```
df = df.withColumn('veg_flag',when(col('Item_Type')=='Meat','Non-Veg').otherwise('Veg'))
```

DECODE THE CODE

`.withColumn(name, expression)`	**Adds** a column, or **replaces** the one with the same name.
`when(condition, value)`	An if / else-if branch for a new column - like SQL CASE WHEN. First match wins.
`col('name')`	Refers to a column by name so you can do something with it (rename, compare, calculate).
`==`	**Equals** test. A single **=** would assign instead of compare.
`.otherwise(value)`	The **else**: used for every row that matched no earlier when().

PYSPARK

```
df.display()
```

ILLUSTRATIVE RESULT

Item_Type	veg_flag
Dairy	Veg
Soft Drinks	Veg
Meat	Non-Veg

If Item_Type is Meat the flag is Non-Veg; **every other row** gets Veg. The result is assigned back, so `df` now has `veg_flag`.

Scenario 2 - several conditions in a chain

PYSPARK

```
df.withColumn('veg_exp_flag',
              when(((col('veg_flag')=='Veg') & (col('Item_MRP')<100)),'Veg_Inexpensive')\
              .when((col('veg_flag')=='Veg') & (col('Item_MRP')>100),'Veg_Expensive')\
              .otherwise('Non_Veg')).display()
```

DECODE THE CODE

`& |`	**AND** / **OR** between conditions. Put each condition in its own ( ).
`\ (backslash)`	Line continuation: the statement carries on in the next line. Nothing may follow it on the same line.

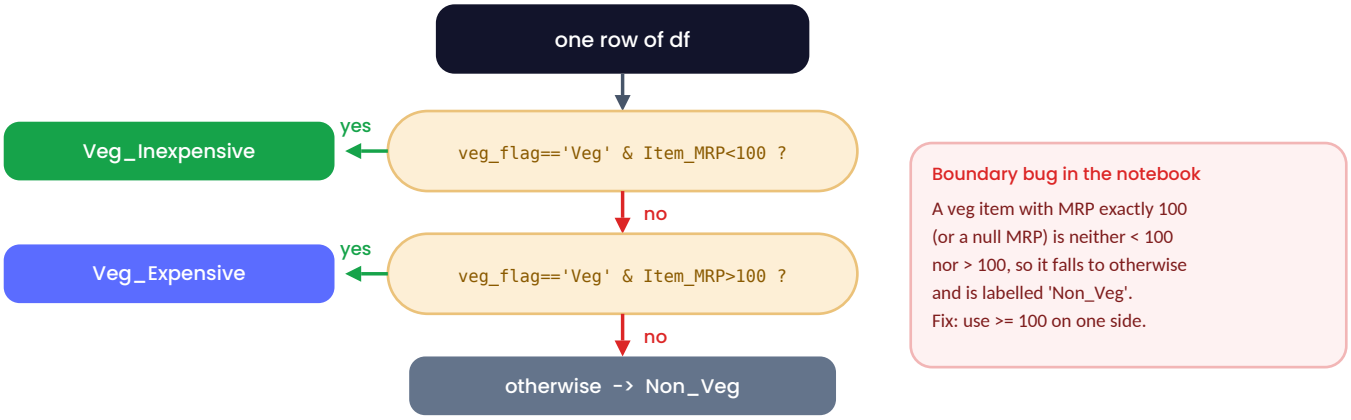
1. Spark tests the first `when`: a veg item with MRP below 100 gets **Veg_Inexpensive**.
2. Only rows that did not match are tested against the second `when`: a veg item with MRP above 100 gets **Veg_Expensive**.
3. Rows matching neither fall to `otherwise`: **Non_Veg**.

Rules: conditions are checked **top to bottom and the first match wins**; combine conditions with `&` / `|` inside parentheses; without `otherwise` unmatched rows become null.

A BOUNDARY BUG IN SCENARIO 2

The two veg conditions are `< 100` and `> 100`. A veg item with an MRP of **exactly 100** matches neither, so it falls through to `otherwise` and is wrongly labelled **Non_Veg**. The same happens to a veg item whose MRP is null.

Fix: make one side inclusive, for example `col('Item_MRP') >= 100` for Veg_Expensive.

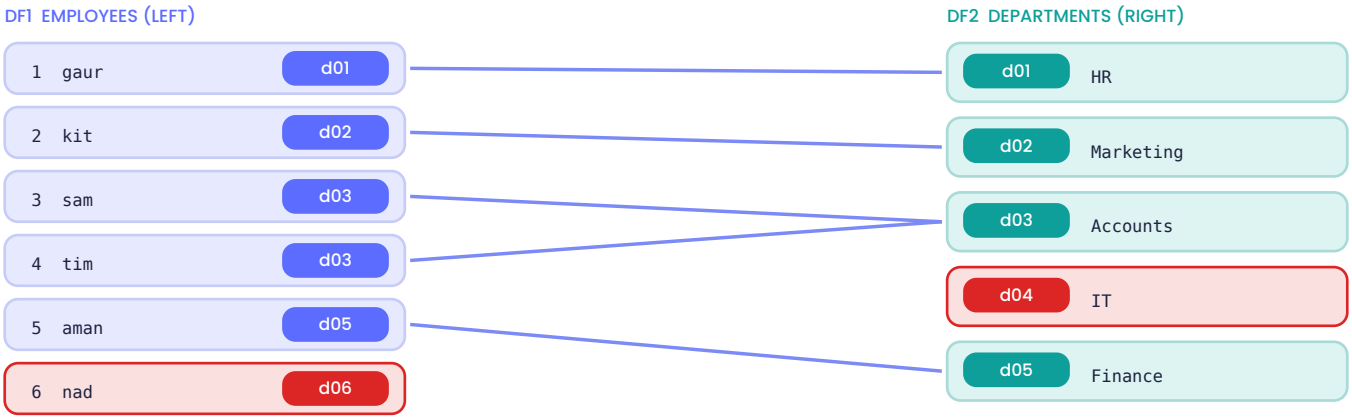


when() conditions are checked top to bottom - the FIRST match wins. otherwise is the else.

`when(...).when(...).otherwise(...)`

4.5 Joins

A **join** combines two DataFrames side by side by matching rows on a shared key - here the department id. The notebook builds an employee table and a department table:



A line = the two rows have the same dept_id (a match). Red = no match: nad (d06) and IT (d04).

PYSPARK

```
dataj1 = [('1','gaur','d01'),
          ('2','kit','d02'),
          ('3','sam','d03'),
          ('4','tim','d03'),
          ('5','aman','d05'),
          ('6','nad','d06')]

schemaj1 = 'emp_id STRING, emp_name STRING, dept_id STRING'

df1 = spark.createDataFrame(dataj1,schemaj1)

dataj2 = [('d01','HR'),
          ('d02','Marketing'),
          ('d03','Accounts'),
          ('d04','IT'),
          ('d05','Finance')]

schemaj2 = 'dept_id STRING, department STRING'

df2 = spark.createDataFrame(dataj2,schemaj2)
```

DECODE THE CODE

`STRING / DOUBLE / INT`	SQL-style type names: text, decimal number, whole number.

df1 (EMPLOYEES)

emp_id	emp_name	dept_id
1	gaur	d01
2	kit	d02
3	sam	d03
4	tim	d03
5	aman	d05
6	nad	d06

df2 (DEPARTMENTS)

dept_id	department
d01	HR
d02	Marketing
d03	Accounts
d04	IT
d05	Finance

> **NOTICE THE GAPS**
> Employee **nad** belongs to **d06**, which does not exist in the departments table. Department **d04 (IT)** has no employees. Each join type treats these two orphans differently - that is the whole point.

Every join uses the same shape: `left_df.join(right_df, condition, how)`.

Inner join - only the matches

PYSPARK

```
df1.join(df2, df1['dept_id']==df2['dept_id'],'inner').display()
```

DECODE THE CODE

`.join(other, condition, how)`	Combines two DataFrames side by side by matching keys.
`df1['dept_id']`	A column taken from one specific DataFrame (useful when both have the same column name).
`'inner' 'left' 'right' 'anti'`	The join type - it decides what happens to rows that find no match.

RESULT: 5 ROWS (nad and IT are dropped)

emp_id	emp_name	dept_id	dept_id	department
1	gaur	d01	d01	HR
2	kit	d02	d02	Marketing
3	sam	d03	d03	Accounts
4	tim	d03	d03	Accounts
5	aman	d05	d05	Finance

Left join - everything from the left, matches from the right

PYSPARK

```
df1.join(df2,df1['dept_id']==df2['dept_id'],'left').display()
```

RESULT: ALL 6 EMPLOYEES (nad has no department)

emp_id	emp_name	dept_id	dept_id	department
1	gaur	d01	d01	HR
2	kit	d02	d02	Marketing
3	sam	d03	d03	Accounts
4	tim	d03	d03	Accounts
5	aman	d05	d05	Finance
6	nad	d06	null	null

Right join - everything from the right, matches from the left

PYSPARK

```
df1.join(df2,df1['dept_id']==df2['dept_id'],'right').display()
```

RESULT: ALL 5 DEPARTMENTS (IT has no employee)

emp_id	emp_name	dept_id	dept_id	department
1	gaur	d01	d01	HR
2	kit	d02	d02	Marketing
3	sam	d03	d03	Accounts
4	tim	d03	d03	Accounts
null	null	null	d04	IT
5	aman	d05	d05	Finance

Anti join - rows with NO match

PYSPARK

```
df1.join(df2,df1['dept_id']==df2['dept_id'],'anti').display()
```

RESULT: LEFT ROWS WITHOUT A MATCH (LEFT COLUMNS ONLY)

emp_id	emp_name	dept_id
6	nad	d06

how =	Keeps	Real-life question
`'inner'`	Rows that match on both sides	Which employees have a valid department?
`'left'`	All left rows; right side null if no match	List every employee, with department if known.
`'right'`	All right rows; left side null if no match	List every department, with employees if any.
`'anti'`	Left rows that have **no** match; left columns only	Which employees have an invalid department?
`'outer'` / `'semi'`	(Extra) all rows from both sides / left rows that **do** match, left columns only	Everything, or an existence check

NOTEBOOK OBSERVATION

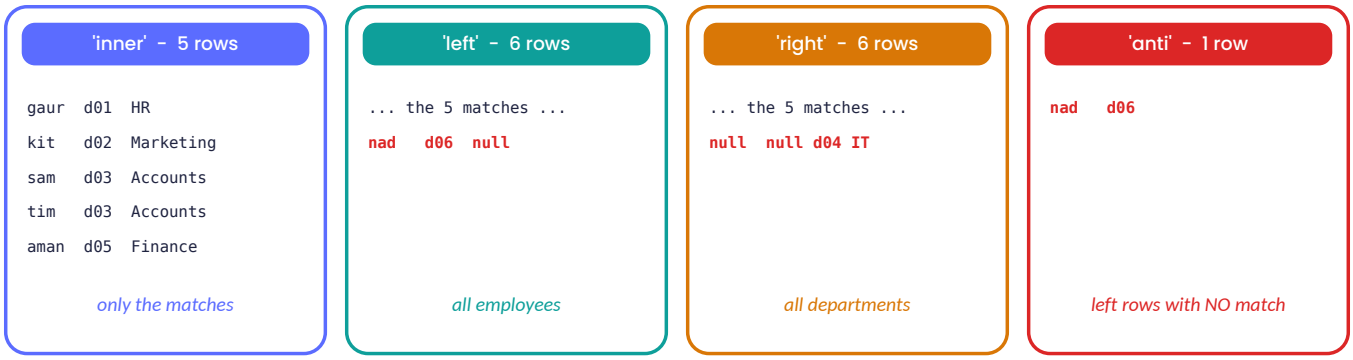
The heading above the right join in the notebook says **LEFT JOIN** (a copy-paste slip). The code beneath it is `'right'`, and that is what it does. The row order of join results is not guaranteed.

DUPLICATE COLUMN NAMES

Joining on `df1['dept_id']==df2['dept_id']` keeps **both** dept_id columns, so a later `select('dept_id')` is ambiguous. Either pick by DataFrame (`df1['dept_id']`), drop one, or use the shortcut below that keeps a single copy.

PYSPARK · EXTRA

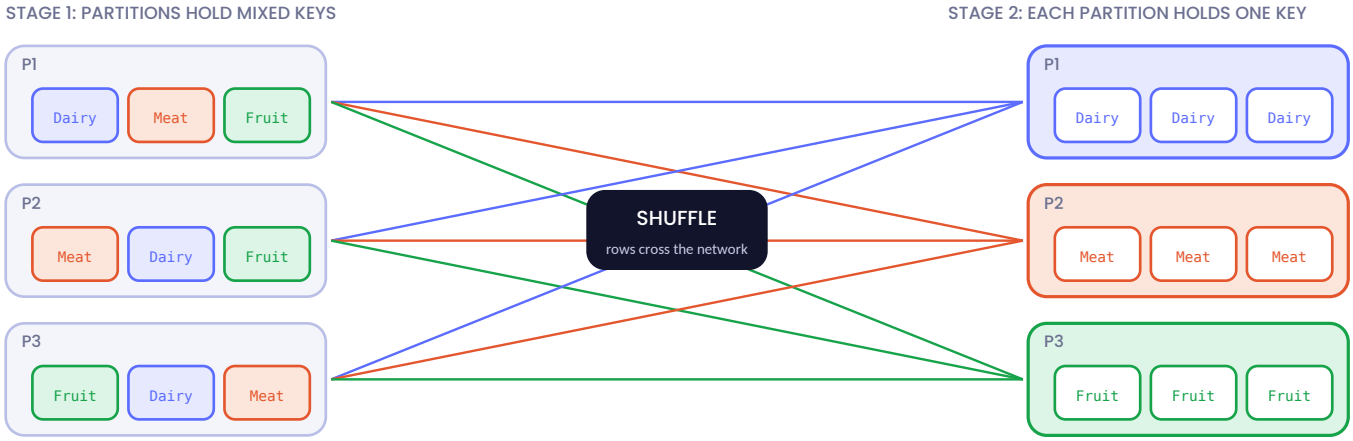
```
df1.join(df2, 'dept_id', 'inner')     # one dept_id column in the result
```



4.6 Shuffles and partitions

BEYOND THE NOTEBOOK

Spark never works on a whole DataFrame at once. It splits it into **partitions**, and every partition is processed by one **task**. More partitions means more tasks that can run side by side - up to the number of cores your executors have.



WHY A SHUFFLE IS EXPENSIVE

Disk write	Network	Serialise	Disk read	Straggler wait
map side saves shuffle files	rows travel between machines	rows packed and unpacked	reduce side fetches + merges	stage ends with slowest task

PYSPARK · HOW MANY PARTITIONS DO I HAVE?

```
df.rdd.getNumPartitions()                          # how many partitions? (not on serverless)
df.select(spark_partition_id()).distinct().count()  # same answer; works on serverless
spark.conf.get('spark.sql.shuffle.partitions')      # partitions AFTER a shuffle (default 200)
```

DECODE THE CODE

`rdd.getNumPartitions()`	How many partitions the DataFrame currently has (not available on serverless).
`spark_partition_id()`	A function giving the partition number of each row - handy to count partitions on serverless.
`.distinct()`	Unique rows across all columns.
`count(col)`	Number of rows / non-null values.
`spark.sql.shuffle.partitions`	How many partitions a shuffle produces (default 200).

OUTPUT · A SMALL CSV READ ON LOCAL SPARK 4.0.1 (a big file gives many more)

```
1
1
200
```

A **shuffle** happens when rows must move so that rows with the same key meet in the same partition: `groupBy`, `join`, `distinct`, `orderBy`, `repartition`. Each shuffle ends a stage (section 0.10). Shuffle data is written to disk by the senders, fetched over the network by the receivers, and the next stage cannot start until every sender is done.

Narrow (no shuffle)	Wide (shuffle)
select, filter, withColumn, cast, when, drop, union	groupBy / agg, join (most), distinct, dropDuplicates, orderBy, window functions, repartition
Each output partition needs **one** input partition.	Each output partition may need rows from **every** input partition.
Cheap: runs in memory, inside one task.	Expensive: disk, network and a stage boundary.

REDUCE THE COST, NOT THE COUNT

Filter and select **before** a wide step so fewer rows move (the optimizer often does this for you).

Prefer one `groupBy` with several aggregates over several separate ones.

A data engineer's habit: look at the plan, count the Exchange lines, and ask whether each one is needed.

200 PARTITIONS FOR 10 ROWS

The default of 200 shuffle partitions suits medium-to-big data. On tiny data it creates many near-empty tasks. Adaptive Query Execution (AQE) merges small shuffle partitions automatically on current Spark and Databricks.

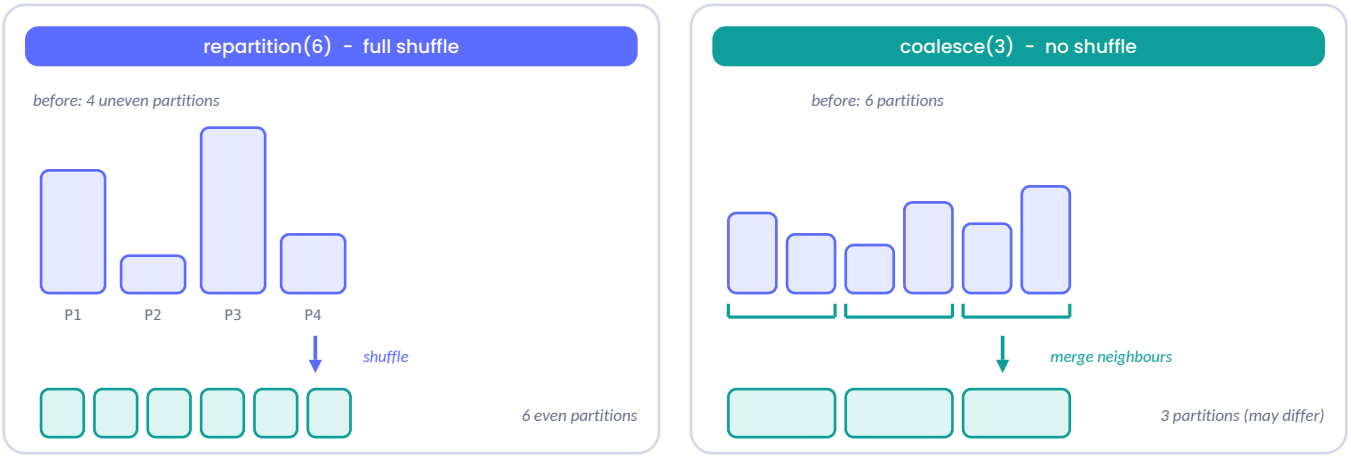
SKEW

If one key holds most of the rows (for example a single store with half the sales), one task does most of the work while the others wait. AQE can split skewed join partitions; otherwise consider salting the key. It is a classic interview topic.

4.7 repartition() vs coalesce()

BEYOND THE NOTEBOOK

Both change the number of partitions. The difference is whether Spark shuffles to do it.



CHOOSING BETWEEN THEM

	repartition(n)	coalesce(n)
Shuffle	Yes - full shuffle	No - merges existing partitions
Direction	Increase or decrease	Decrease only
Partition sizes	Evened out	Can stay uneven
Typical use	More parallelism, fix skew, group by key	Fewer output files before a write

PYSPARK · CHANGE THE NUMBER OF PARTITIONS

```
d8 = df.repartition(8)                  # 8 partitions, evenly balanced, FULL shuffle
dk = df.repartition('Item_Type')        # rows with the same Item_Type go to the same partition
d2 = d8.coalesce(2)                     # 2 partitions by merging neighbours, NO shuffle
print(d8.rdd.getNumPartitions(), d2.rdd.getNumPartitions())
```

DECODE THE CODE

`.repartition(n | col)`	Reshuffles the data into **n** balanced partitions (or by a column). Can increase or decrease.
`.coalesce(n)`	Reduces the partitions by merging neighbours - **no** shuffle. Cannot increase.

OUTPUT · (Spark 4.0.1)

```
8 2
```

	repartition(n)	coalesce(n)
Can increase partitions?	Yes	**No** - it only reduces
Shuffle?	Yes (full)	No - merges existing partitions
Partition sizes	Balanced	Can be uneven
Typical use	More parallelism; spread skewed data; partition by a column before a write or join	Fewer, bigger output files just before a write

PYSPARK · ONE OUTPUT FILE (SMALL DATA ONLY)

```
(df.coalesce(1).write.format('csv').mode('overwrite').option('header', True)
   .save('/Volumes/main/demo/out/one_file'))
```

DECODE THE CODE

`df.write`	Starts a **DataFrameWriter** - the mirror image of spark.read.
`.format('csv')`	Chooses the file format: **csv**, **json**, **parquet** ... (the same call is used when writing).
`.mode('append')`	What to do if the destination already exists: error, append, overwrite or ignore.
`.option(key, value)`	Sets **one** setting for the reader or writer. Call it again for each extra setting.
`header`	**True** = the first row of the CSV holds column names, not data.
`.save(path)`	Performs the write. This is the **action** that makes the job run.

COALESCE(1) IS A BOTTLENECK

It forces the whole write through a single task. Fine for a small report; a mistake for large data. Remember from Phase 6 that Spark still writes a **folder** with one part file, not a single named file.

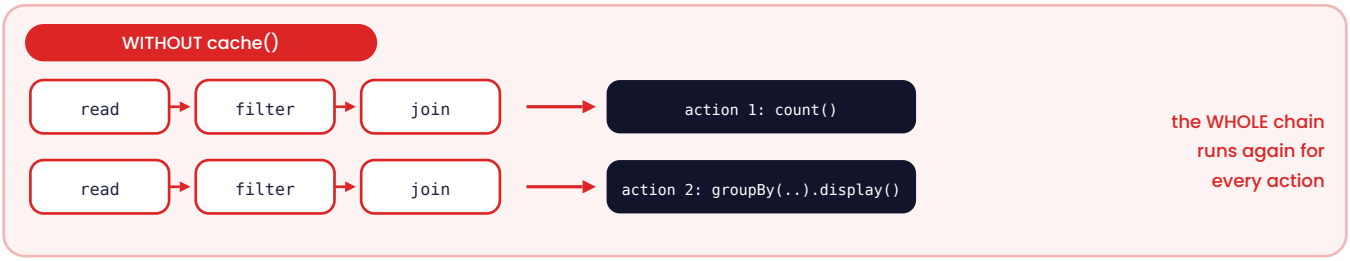
FILES PER WRITE

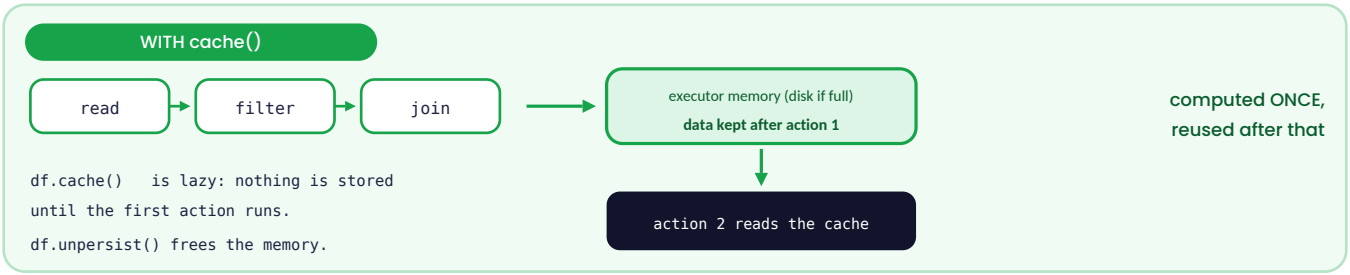
The number of partitions at write time is the number of output files (per folder). Too many tiny files slow every later read; too few huge files cannot be read in parallel. Delta's `OPTIMIZE` (Phase 6) can compact small files later.

4.8 cache() vs persist()

BEYOND THE NOTEBOOK

Because Spark is lazy, a DataFrame you use twice is **computed twice**. `cache()` asks Spark to keep the result after the first computation so the second use reads it from memory.





PYSPARK · CACHE A DataFrame YOU REUSE

```
big = df.filter(col('Item_MRP') > 50)
big.cache()                 # marks it for caching - still lazy, nothing happens yet
big.count()                 # first action: computes AND stores
big.groupBy('Item_Type').count().display()      # reuses the stored copy
big.unpersist()             # release the memory when you are done
```

DECODE THE CODE

`.filter(condition)`	Keeps the rows where the condition is true - like SQL WHERE.
`.cache()`	Keeps the computed DataFrame for reuse (lazy: stored on the first action). Not on serverless.
`.unpersist()`	Frees the cached copy.

PYSPARK · persist() LETS YOU CHOOSE WHERE

```
from pyspark import StorageLevel
big.persist(StorageLevel.MEMORY_AND_DISK)    # the same idea, with a chosen storage level
```

DECODE THE CODE

`StorageLevel`	Where persisted data is kept: *MEMORY_ONLY*, *MEMORY_AND_DISK*, *DISK_ONLY* ...
`.persist(level)`	Like cache, but you choose the storage level (memory, disk, or both).

	cache()	persist(level)
What it is	Shortcut for persist with the default level	You pick the storage level
Default level (DataFrame)	MEMORY_AND_DISK	whatever you pass
Typical levels	-	MEMORY_ONLY, MEMORY_AND_DISK, DISK_ONLY
Is it lazy?	Yes - nothing is stored until an action runs	Yes

WHEN CACHING HELPS

The same DataFrame is used by **several actions** or branches, and recomputing it is expensive (a big join, a long chain).

Do **not** cache something used once, or something so large that it will not fit - you only add overhead.

Always `unpersist()` when finished.

NOT AVAILABLE ON SERVERLESS

Databricks documents that `cache()`, `persist()` and `unpersist()` are not supported on serverless compute (which is all of Free Edition). There, write an intermediate result to a Delta table instead, which is also safer for pipelines. On classic clusters the code above works as shown.

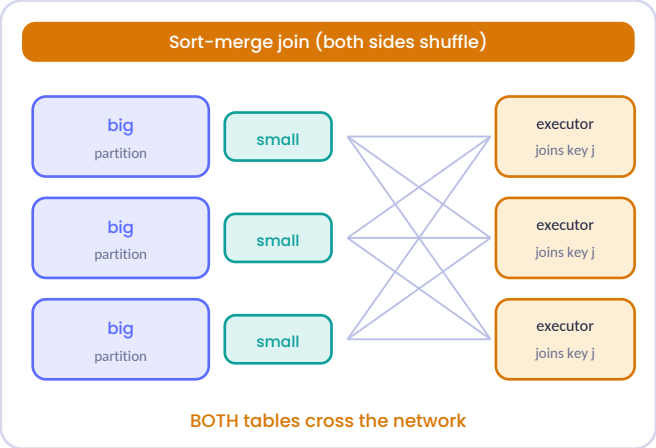
CACHED OR JUST TEMPORARY?

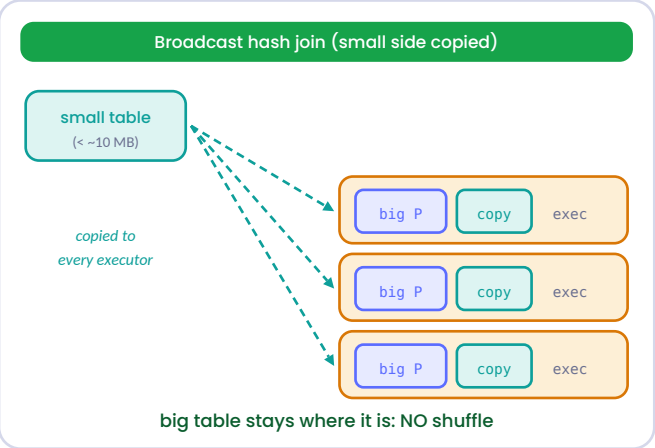
Phase 7's `createTempView` does **not** cache anything - it is only a name. Each query on the view recomputes the DataFrame unless you cache it.

4.9 Broadcast joins and join strategies

BEYOND THE NOTEBOOK

Section 4.5 showed *what* a join returns. Here is *how* Spark runs it. A **sort-merge join** shuffles **both** sides. A **broadcast hash join** sends a copy of the small side to every executor, so the big side never moves.






```
from pyspark.sql.functions import broadcast
df1.join(broadcast(df2), df1['dept_id'] == df2['dept_id'], 'inner')
# spark.sql.autoBroadcastJoinThreshold = 10 MB by default; AQE can also switch to a broadcast at run time
```

PYSPARK · HINT THAT THE SMALL TABLE SHOULD BE BROADCAST

```
joined = df1.join(broadcast(df2), df1['dept_id'] == df2['dept_id'], 'inner')
joined.explain()
```

DECODE THE CODE

`broadcast(df)`	Hint: copy this small DataFrame to every executor so the join needs no shuffle of the big side.
`.explain()`	Prints the plan Spark will run - **without** running it. *mode='formatted'* gives an easier outline.

OUTPUT · SIMPLIFIED (ids like #2 removed) – NO Exchange hashpartitioning ON THE BIG SIDE

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- BroadcastHashJoin [dept_id], [dept_id], Inner, BuildRight
   :- Filter isnotnull(dept_id)
   :  +- Scan ExistingRDD[emp_id,emp_name,dept_id]
   +- BroadcastExchange HashedRelationBroadcastMode(...)
      +- Filter isnotnull(dept_id)
         +- Scan ExistingRDD[dept_id,department]
```

Without the hint, the same join on these two tiny in-memory tables ran as a **SortMergeJoin**, with two `Exchange hashpartitioning(dept_id, 200)` lines. Spark broadcasts automatically only when it *knows* a side is smaller than `spark.sql.autoBroadcastJoinThreshold` (10 MB by default); a DataFrame built from Python lists carries no size statistics, so it chose the safe plan.

Strategy	Moves the big side?	Use when
BroadcastHashJoin	No	One side is small (fits comfortably in each executor's memory).
SortMergeJoin	Yes - both sides shuffle and sort	Both sides are large. The default for big joins.
Shuffle hash join	Yes	One side is moderately smaller; chosen less often.

BROADCAST ONLY WHAT IS REALLY SMALL

A broadcast table is held in memory on the driver and on **every** executor. Broadcasting something big can fail with out-of-memory. Use it for dimension tables such as departments, stores or product categories.

JOINS THAT MATTER FOR CORRECTNESS

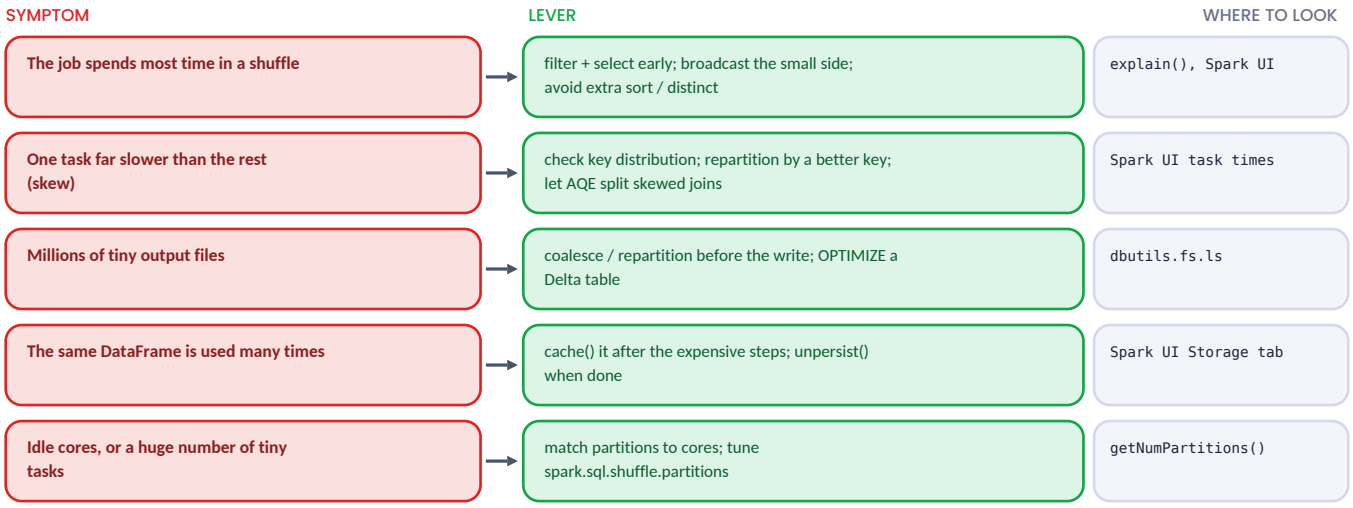
A join key that is **not unique** on the small side multiplies rows (see 4.11).

Join on the **same data type** on both sides, or Spark may cast one side and lose the benefit of pruning.

4.10 A basic tuning checklist

BEYOND THE NOTEBOOK

Tuning is a loop: **look at the plan, find the cost, change one thing, measure again**. For a beginner, five checks cover most cases.



#	Question	Fix
1	Am I reading more than I need?	Select only the needed columns and filter early. Prefer Parquet / Delta over CSV.
2	How many shuffles (Exchange lines)?	Remove avoidable ones; combine aggregations; filter before wide steps.
3	Is a small table being shuffled in a join?	Broadcast it (`broadcast(df2)`).
4	Too many or too few partitions?	`repartition` for more parallelism, `coalesce` for fewer output files.
5	Am I recomputing the same thing?	`cache()` (classic clusters) or save an intermediate Delta table.
6	Is one task slower than the rest?	Suspect **skew**: check key distribution; AQE skew handling or salting.

READ, THEN CHANGE

Use `explain()` first, and the Spark UI (classic) or the query profile (serverless) to see time per stage. A change without a measurement is a guess.

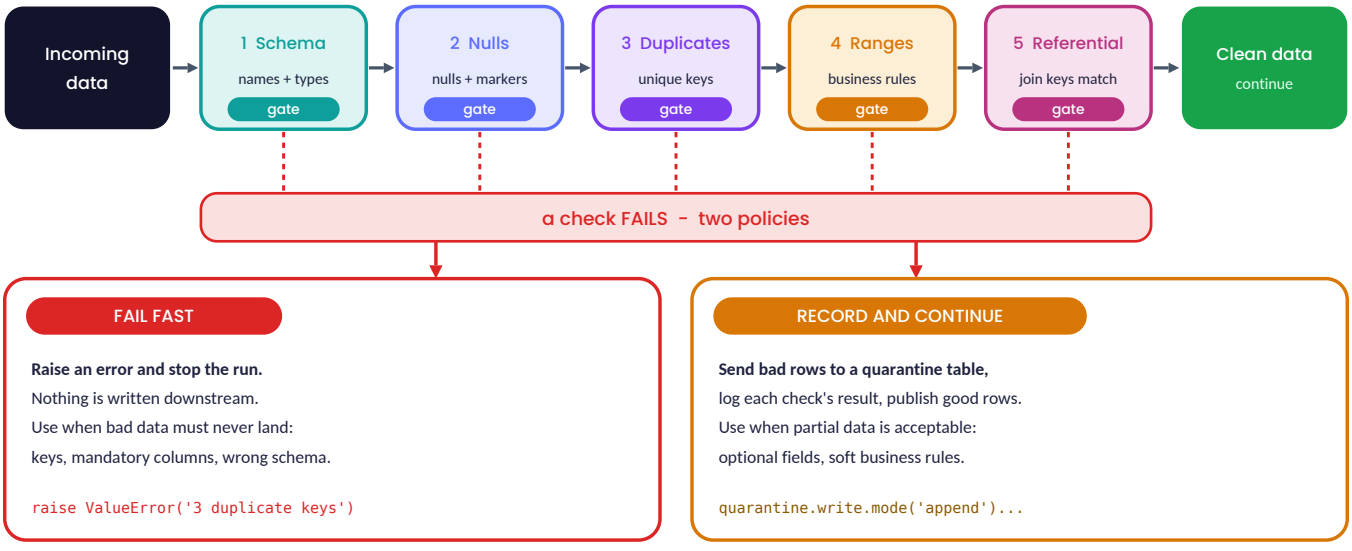
NOT A PERFORMANCE LECTURE

These notes only introduce the vocabulary. Deep tuning - memory, spill, join hints, file sizing - comes with real data volumes and real clusters.

4.11 Data-quality checks

BEYOND THE NOTEBOOK

A pipeline that runs without errors can still produce **wrong** data. Data-quality (DQ) checks are small, explicit tests on the data. These examples use the notebook's tables: `df1` (employees) and `df2` (departments). Employee 6 works in **d06**, a department that does not exist, and department **d04** has no employees.



Most pipelines mix both: a few critical checks fail fast, the rest are recorded and reviewed.

Check	Question it answers	Example on the BigMart data
Completeness	Are required values present?	Item_Identifier is never null.
Uniqueness	Is the key unique?	One row per Item_Identifier + Outlet_Identifier.
Validity / range	Are values inside business rules?	Item_Visibility between 0 and 1; Item_MRP > 0.
Domain / set	Is the value one that is allowed?	Item_Fat_Content in (Lf, Regular).
Referential integrity	Does every foreign key exist in the parent?	Every dept_id in employees exists in departments.
Consistency	Do related columns agree?	Outlet_Establishment_Year not in the future.

Duplicate keys

PYSPARK · WHICH KEYS APPEAR MORE THAN ONCE?

```
df2_dupes = df2.groupBy('dept_id').count().filter(col('count') > 1)
df1_dupes = df1.groupBy('dept_id').count().filter(col('count') > 1)
df1_dupes.show()
```

DECODE THE CODE

`.show(n, truncate)`	Prints the first rows as text in any Spark environment; *truncate=False* stops long values being cut.

OUTPUT · d03 repeats in EMPLOYEES (fine: many employees per department); departments has no repeats

dept_id	count
d03	2

DUPLICATES ARE ONLY WRONG IN THE RIGHT PLACE

A key must be unique in the **parent** (departments), but it is normal for it to repeat in the **child** (employees). The check is always about the side where you expect one row per key.

Range and business rules

PYSPARK · ROWS THAT BREAK A RULE

```
bad_vis  = df.filter(~col('Item_Visibility').between(0, 1))
bad_year = df.filter(~col('Outlet_Establishment_Year').between(1950, 2026))
bad_fat  = df.filter(~col('Item_Fat_Content').isin('Lf', 'Regular'))
```

DECODE THE CODE

`.between(low, high)`	True when the value is from low to high, **inclusive**. Null gives null.
`.isin(a, b)`	True when the value is in the given list.

NULLS AND NEGATIVE FILTERS

A comparison with null is null, so `~col('x').between(0,1)` does **not** return rows where x is null. Add `| col('x').isNull()` if a missing value should also fail.

Referential integrity: find unmatched join keys

PYSPARK · left_anti KEEPS ONLY THE ROWS WITH NO MATCH

```
orphans = df1.join(df2, 'dept_id', 'left_anti')     # employees with an unknown dept_id
unused  = df2.join(df1, 'dept_id', 'left_anti')     # departments with no employees
orphans.show()
unused.show()
```

DECODE THE CODE

`'left_anti'`	Join type: keep left rows that have **no** match on the right. Only left columns come back.

orphans · EMPLOYEES WITH NO DEPARTMENT

dept_id	emp_id	emp_name
d06	6	nad

unused · DEPARTMENTS NOBODY BELONGS TO

dept_id	department
d04	IT

A **left join** followed by a null test finds the same orphans, but also returns the department columns. `left_anti` is shorter and cannot multiply rows.

PYSPARK · THE LEFT-JOIN WAY

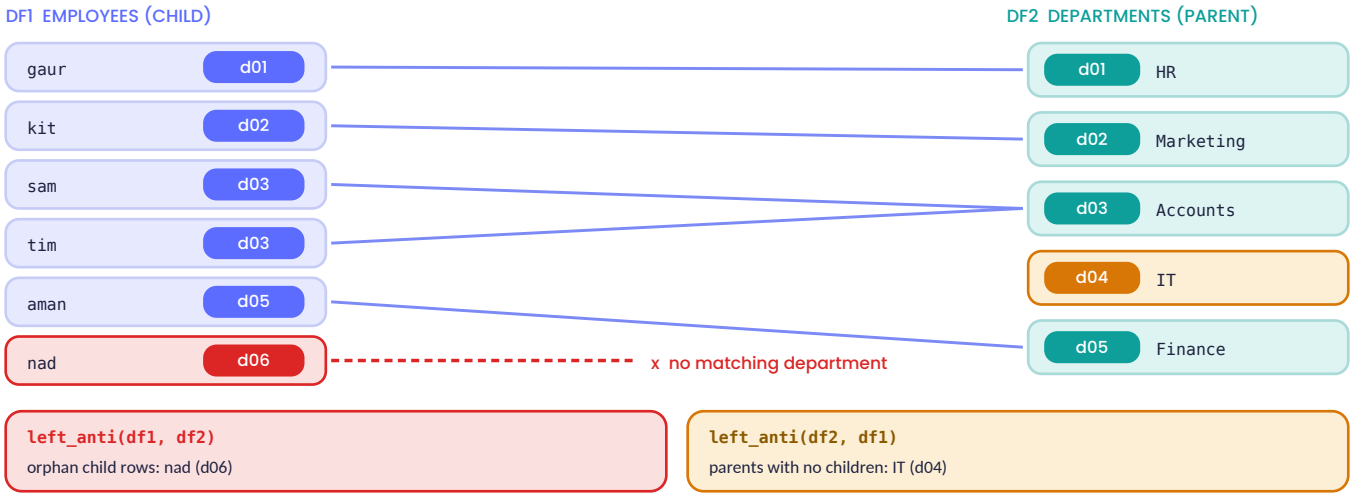
```
j = df1.join(df2, 'dept_id', 'left')
j.filter(col('department').isNull()).show()                # same orphan, via a left join
print(df1.count(), j.count())                              # 6 6 - a left join keeps every employee
```

DECODE THE CODE

`.isNull()`	True where the value is missing. (A null is never *equal* to anything, so == *None* fails.)

INNER JOIN HIDES THE PROBLEM

An inner join simply **drops** employee 6 and no error appears - a silent loss of data. After every important join, compare row counts or run an anti join.



Join fan-out: when a join creates extra rows

PYSPARK · A DUPLICATE KEY IN THE PARENT

```
extra = spark.createDataFrame([('d03', 'Accounts-old')], 'dept_id STRING, department STRING')
df2b = df2.union(extra)
print(df1.count(), df1.join(df2b, 'dept_id', 'left').count())
```

DECODE THE CODE

`.union(other)`	Stacks the rows of two DataFrames, matching columns by **position**.

OUTPUT · TWO EMPLOYEES IN d03 x TWO d03 ROWS = 4, SO 6 BECOMES 8

```
6 8
```

ALWAYS CHECK THE PARENT'S KEY

Fan-out inflates totals (revenue counted twice) without an error. Check uniqueness of the join key on the dimension side **before** joining.

LEFT_SEMI

`left_semi` is the opposite of anti: it keeps rows that **do** have a match, returns only the left columns, and never duplicates rows: `df1.join(df2.select('dept_id').distinct(), 'dept_id', 'left_semi')` returns the 5 employees with a valid department.

4.12 Reusable checks, fail-fast vs recording failures

BEYOND THE NOTEBOOK

Write each check once as a small function that **returns the failing rows**. Then every check works the same way: empty result = pass.

PYSPARK · A SMALL REUSABLE CHECK LIBRARY

```
from pyspark.sql import functions as F

def check_not_null(df, cols):
    cond = F.lit(False)
    for c in cols:
        cond = cond | F.col(c).isNull()
    return df.filter(cond)

def check_unique(df, keys):
    return df.groupBy(keys).count().filter(F.col('count') > 1)

def check_range(df, c, low=None, high=None):
    ok = F.col(c).isNotNull()
    if low is not None:
        ok = ok & (F.col(c) >= low)
    if high is not None:
        ok = ok & (F.col(c) <= high)
    return df.filter(~ok)

def check_in_set(df, c, allowed):
    return df.filter(~F.col(c).isin(allowed) | F.col(c).isNull())

def check_orphans(child, parent, key):
    return child.join(parent.select(key).distinct(), key, 'left_anti')
```

DECODE THE CODE

`functions as F`	Imports Spark's functions under the short name **F**, so you write *F.col*, *F.sum* …
`def name(args):`	Defines an ordinary Python function.
`lit(value)`	Turns a plain Python value into a constant column (otherwise Spark looks for a column with that name).
`.isNotNull()`	True when the value is not null.

PYSPARK · RUN THEM ALL AND BUILD A REPORT

```
def run_checks(df, checks):
    rows = []
    for name, fn in checks:
        failed = fn(df).count()                    # an action: runs the check
        rows.append((name, 'PASS' if failed == 0 else 'FAIL', failed))
    return spark.createDataFrame(rows, 'check STRING, status STRING, failed_rows BIGINT')

checks = [
    ('emp_id is not null',            lambda d: check_not_null(d, ['emp_id'])),
    ('emp_id is unique',              lambda d: check_unique(d, ['emp_id'])),
    ('dept_id exists in departments', lambda d: check_orphans(d, df2, 'dept_id')),
]
report = run_checks(df1, checks)
report.show(truncate=False)
```

DECODE THE CODE

`lambda x: ...`	A tiny unnamed Python function written inline.

OUTPUT · (Spark 4.0.1)

```
+----------------------------+------+-----------+
|check                       |status|failed_rows|
+----------------------------+------+-----------+
|emp_id is not null          |PASS  |0          |
|emp_id is unique            |PASS  |0          |
|dept_id exists in departments|FAIL  |1          |
+----------------------------+------+-----------+
```

Fail-fast vs record-and-continue

	Fail fast	Record the failure and continue
What happens	The pipeline stops with an error.	Bad rows go to a quarantine table; good rows move on.
Best for	**Critical** problems: missing key column, wrong schema, duplicate primary key, nothing loaded.	**Row-level** problems you can live with and fix later: bad range, unknown department.
Risk	One bad row can block a whole load.	Bad data may go unnoticed if nobody reads the quarantine.
Also needs	A clear error message and an alert.	A log of which check failed, when, and for which rows.

PYSPARK · FAIL FAST ON THE CRITICAL CHECKS

```
def fail_on(report, critical):
    bad = report.filter((F.col('status') == 'FAIL') & F.col('check').isin(critical)).collect()
    if bad:
        raise ValueError('Critical checks failed: ' + ', '.join(r['check'] for r in bad))

fail_on(report, ['emp_id is unique'])                       # passes silently
fail_on(report, ['dept_id exists in departments'])          # raises ValueError
```

DECODE THE CODE

`.collect()`	Brings all result rows to the driver as a Python list - an action; only for small results.
`raise`	Throws an error. Uncaught, it stops the notebook / marks the job Failed.

OUTPUT · FROM THE SECOND CALL

```
ValueError: Critical checks failed: dept_id exists in departments
```

PYSPARK · SEND THE BAD ROWS TO A QUARANTINE TABLE

```
bad_rows = check_orphans(df1, df2, 'dept_id')
quarantine = (bad_rows.withColumn('failed_check', F.lit('dept_id exists in departments'))
                      .withColumn('checked_at', F.current_timestamp()))
quarantine.write.mode('append').saveAsTable('dq_quarantine')

good = df1.join(df2.select('dept_id').distinct(), 'dept_id', 'left_semi')   # rows that may go on
```

DECODE THE CODE

`current_timestamp()`	The current date and time - used to stamp rows with a load time.
`.saveAsTable('name')`	Saves the data as a **named table** in Spark's catalog.
`'left_semi'`	Join type: keep left rows that **do** have a match. Never duplicates rows.

dq_quarantine · ONE ROW, WITH THE REASON AND A TIMESTAMP

dept_id	emp_id	emp_name	failed_check	checked_at
d06	6	nad	dept_id exists in departments	2026-10-16 06:47:...

COUNT ONLY ONCE

Each `.count()` is a separate job. On big data run the checks on a cached or saved DataFrame (4.8), or compute several checks in one `.agg(...)` to avoid scanning the data again and again.

WHERE EACH CHECK BELONGS

Schema and key checks at the **Bronze** boundary, business rules and referential checks before data enters **Silver**, and a final reconciliation (row counts, totals) before **Gold** (Phase 7).

INTERVIEW CORNER

Q What is a shuffle and why does it matter?

A A redistribution of rows across partitions so that rows with the same key are together. It is triggered by wide transformations, splits a job into stages, and is costly because of disk and network.

Q repartition or coalesce?

A repartition can increase or decrease partitions and does a full shuffle (balanced result). coalesce only decreases, merges neighbouring partitions and avoids a shuffle - best for fewer output files.

Q cache or persist?

A cache() is persist() with the default storage level. Use persist(level) to choose memory, disk or both. Both are lazy and should be released with unpersist(); they are not supported on serverless.

Q What is a broadcast join and when do you use it?

A The small table is copied to every executor so the large table is joined without shuffling. Use it for small dimension tables; the broadcast limit defaults to 10 MB.

Q A job has one slow task. What do you suspect?

A Data skew: one key has far more rows. Check the key distribution, rely on AQE skew handling, or salt the key.

Q How would you find rows in the fact table that have no match in the dimension?

A A left_anti join of the fact table with the distinct dimension keys - or a left join and a filter for null on a dimension column.

Q Your totals doubled after a join. Why?

A Fan-out: the join key is not unique on one side, so rows multiplied. Check key uniqueness (groupBy + count > 1) before joining.

Q Fail fast or record failures?

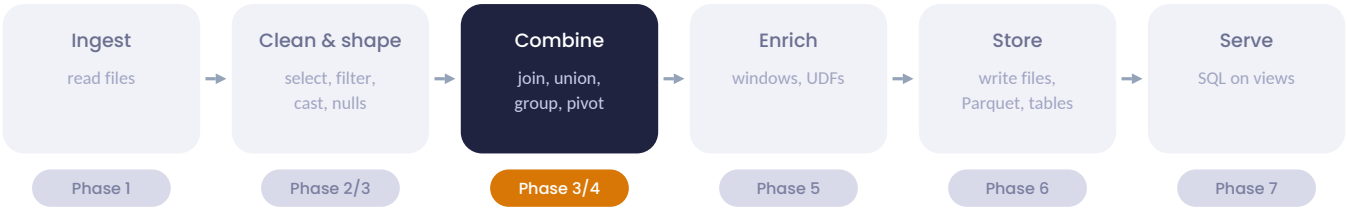
A Fail fast on critical, structural problems (schema, key, empty load); record row-level problems in a quarantine table with the reason, and report them.

PHASE 4 CHECKPOINT

- I can group by one or many columns and compute sum, avg and others, naming results with alias.
- I can gather values into lists with collect_list and reshape rows into columns with pivot.
- I can build if / else-if / else columns with when and otherwise, and I watch the boundaries.
- I can run inner, left, right and anti joins and predict which rows each returns.
- I know a join on same-named columns leaves two columns with the same name.
- I can explain partitions, shuffles and why wide transformations create stages.
- I can choose between `repartition` and `coalesce`, and between `cache` and `persist`.
- I can read a join plan and say when a broadcast join is the right choice.
- I can write reusable checks for nulls, duplicates, ranges, allowed values and orphan keys, and decide between fail-fast and quarantine.

From notebook to pipeline

Data engineers use these same skills inside **pipelines**. Here is where this phase sits, and how it is used in practice.



DATA ENGINEERING LENS

Aggregations build summary tables: total or average per group, with alias names that stay readable downstream.

Joins enrich one table with another (employees get a department name). The join type decides what to keep: use left to keep every employee, inner to keep only matches.

anti join is a quick data-quality check: it returns the rows whose key has no match (employee nad with the unknown department d06).

After a join on same-named columns you have two dept_id columns - drop one or join on the column name.

Beyond the notebook: after every important join check the row count and run an **anti join**; an inner join silently drops unmatched rows and a duplicate parent key silently multiplies them.

Fail fast on critical checks (schema, key); quarantine row-level failures with the reason and a timestamp.

Glossary

Concepts from this phase, then every **code term** the decoder explained, in the order you met them.

Concept	In plain words
Aggregation	Many rows summarised into one value (sum, avg ...).
Group	The set of rows sharing the same key value(s).
Pivot	Turning the distinct values of a column into column headings.
Join key	The column(s) used to match rows of two DataFrames.
Inner / left / right / anti join	Keep matches only / all left rows / all right rows / left rows with no match.
CASE WHEN	SQL's if / else-if / else, written when / otherwise in PySpark.
Skew	One key holds far more rows than the others, so one task runs much longer.
Broadcast join	The small table is copied to every executor so the big table does not shuffle.
repartition / coalesce	Change the partition count with a full shuffle / by merging, without a shuffle.
cache / persist	Keep a computed DataFrame in memory (and disk) for reuse.
Referential integrity	Every foreign key has a matching row in the parent table.
Quarantine table	Where rows that failed a check are saved, with the reason.

Code terms decoded in this phase

Code term	Meaning
`.display()`	Databricks' table viewer. It is an **action**, so it makes Spark run the job.
`.groupBy(columns)`	Collects rows that share the same key(s) into groups. Needs an aggregate next.
`.agg(...)`	Calculates one or more aggregates for every group.
`sum(col)`	Total of a column (Spark's own *sum*, available through the import *).
`avg(col)`	Average of a column.
`.alias('new')`	Gives a column a new name **in this result only**.
`[('1','kad'), ...]`	A Python list of tuples. Each tuple is one row of the DataFrame.
`spark`	The SparkSession - your entry point to Spark. Databricks creates it for you, already named **spark**.
`spark.createDataFrame(rows, schema)`	Builds a DataFrame from Python data.
`spark.createDataFrame(data, schema)`	Builds a small DataFrame from a Python list plus a schema string.
`collect_list(col)`	Gathers a group's values into one **array** (the opposite of explode).
`.select(...)`	Keeps only the listed columns (or new calculated columns) - like SQL SELECT.
`collect_set(col)`	Like collect_list but without repeated values.
`.pivot(col)`	Turns the distinct values of a column into **column headings**.
`.withColumn(name, expression)`	**Adds** a column, or **replaces** the one with the same name.
`when(condition, value)`	An if / else-if branch for a new column - like SQL CASE WHEN. First match wins.
`col('name')`	Refers to a column by name so you can do something with it (rename, compare, calculate).
`==`	**Equals** test. A single **=** would assign instead of compare.
`.otherwise(value)`	The **else**: used for every row that matched no earlier when().
`& |`	**AND** / **OR** between conditions. Put each condition in its own ( ).
`\ (backslash)`	Line continuation: the statement carries on in the next line. Nothing may follow it on the same line.
`STRING / DOUBLE / INT`	SQL-style type names: text, decimal number, whole number.
`.join(other, condition, how)`	Combines two DataFrames side by side by matching keys.
`df1['dept_id']`	A column taken from one specific DataFrame (useful when both have the same column name).
`'inner' 'left' 'right' 'anti'`	The join type - it decides what happens to rows that find no match.
`rdd.getNumPartitions()`	How many partitions the DataFrame currently has (not available on serverless).
`spark_partition_id()`	A function giving the partition number of each row - handy to count partitions on serverless.

Code term	Meaning
`.distinct()`	Unique rows across all columns.
`count(col)`	Number of rows / non-null values.
`spark.sql.shuffle.partitions`	How many partitions a shuffle produces (default 200).
`.repartition(n | col)`	Reshuffles the data into **n** balanced partitions (or by a column). Can increase or decrease.
`.coalesce(n)`	Reduces the partitions by merging neighbours - **no** shuffle. Cannot increase.
`df.write`	Starts a **DataFrameWriter** - the mirror image of spark.read.
`.format('csv')`	Chooses the file format: **csv**, **json**, **parquet** ... (the same call is used when writing).
`.mode('append')`	What to do if the destination already exists: error, append, overwrite or ignore.
`.option(key, value)`	Sets **one** setting for the reader or writer. Call it again for each extra setting.
`header`	**True** = the first row of the CSV holds column names, not data.
`.save(path)`	Performs the write. This is the **action** that makes the job run.
`.filter(condition)`	Keeps the rows where the condition is true - like SQL WHERE.
`.cache()`	Keeps the computed DataFrame for reuse (lazy: stored on the first action). Not on serverless.
`.unpersist()`	Frees the cached copy.
`StorageLevel`	Where persisted data is kept: *MEMORY_ONLY*, *MEMORY_AND_DISK*, *DISK_ONLY* ...
`.persist(level)`	Like cache, but you choose the storage level (memory, disk, or both).
`broadcast(df)`	Hint: copy this small DataFrame to every executor so the join needs no shuffle of the big side.
`.explain()`	Prints the plan Spark will run - **without** running it. *mode='formatted'* gives an easier outline.
`.show(n, truncate)`	Prints the first rows as text in any Spark environment; *truncate=False* stops long values being cut.
`.between(low, high)`	True when the value is from low to high, **inclusive**. Null gives null.
`.isin(a, b)`	True when the value is in the given list.
`'left_anti'`	Join type: keep left rows that have **no** match on the right. Only left columns come back.
`.isNull()`	True where the value is missing. (A null is never *equal* to anything, so == *None* fails.)
`.union(other)`	Stacks the rows of two DataFrames, matching columns by **position**.
`functions as F`	Imports Spark's functions under the short name **F**, so you write *F.col*, *F.sum* ...
`def name(args):`	Defines an ordinary Python function.
`lit(value)`	Turns a plain Python value into a constant column (otherwise Spark looks for a column with that name).
`.isNotNull()`	True when the value is not null.
`lambda x: ...`	A tiny unnamed Python function written inline.

Code term	Meaning
`.collect()`	Brings all result rows to the driver as a Python list - an action; only for small results.
`raise`	Throws an error. Uncaught, it stops the notebook / marks the job Failed.
`current_timestamp()`	The current date and time - used to stamp rows with a load time.
`.saveAsTable('name')`	Saves the data as a **named table** in Spark's catalog.
`'left_semi'`	Join type: keep left rows that **do** have a match. Never duplicates rows.

Check yourself

Try each question first, then click **Show answer**. Every answer has a **back** link that returns you to the question.

Questions

Q1 What does `groupBy('Item_Type','Outlet_Size').agg(sum('Item_MRP'))` return? Show answer

Q2 What is `collect_list` for? Show answer

Q3 Which join lists the employees whose department does not exist, and what does it return for the notebook's data? Show answer

Q4 How many rows do the inner, left, right and anti joins return for the notebook's employee and department tables? Show answer

Q5 Why is a veg item with MRP exactly 100 labelled Non_Veg in the notebook? Show answer

Q6 Which of repartition and coalesce can increase the number of partitions, and which avoids a shuffle? Show answer

Q7 When is a broadcast join the right choice? Show answer

Q8 After a left join the totals doubled. What do you check? Show answer

Q9 Which join finds employees whose department does not exist, without adding columns? Show answer

Hands-on tasks

Task 1 Total Item_MRP per Outlet_Type as Total_MRP, highest first. Show solution

Task 2 Using the employee (df1) and department (df2) tables, find the employees without a valid department using a LEFT join instead of an anti join. Show solution

Task 3 Find the departments that have no employees, then check that the join key is unique in the departments table. Show solution

Answers and solutions

A1 One row for each **combination** of Item_Type and Outlet_Size, with the total Item_MRP of that group. back to question

A2 It gathers all the values of a group into one array. It must follow a groupBy. back to question

A3 The **anti** join: `df1.join(df2, df1['dept_id']==df2['dept_id'], 'anti')` returns one row - nad (d06). back to question

A4 Inner 5, left 6, right 6 (including IT with no employees), anti 1. back to question

A5 The conditions are `< 100` and `> 100`, so 100 matches neither and falls to `otherwise('Non_Veg')`. Make one side inclusive (>=). back to question

A6 repartition can increase (and does a full shuffle); coalesce only decreases and avoids the shuffle. back to question

A7 When one side is small (below about 10 MB by default): it is copied to every executor so the large side is not shuffled. back to question

A8 Whether the join key is **unique on the dimension side**: a duplicate key multiplies rows (fan-out). back to question

A9 A `left_anti` join of the employees with the departments on dept_id. back to question

Task solutions

Solution 1 back to task

SOLUTION

```
df.groupBy('Outlet_Type')\
  .agg(sum('Item_MRP').alias('Total_MRP'))\
  .sort(col('Total_MRP').desc())\
  .display()
```

Solution 2 back to task

SOLUTION

```
df1.join(df2, df1['dept_id'] == df2['dept_id'], 'left')\
   .filter(df2['department'].isNull())\
   .display()      # returns nad
```

Solution 3 back to task

SOLUTION

```
df2.join(df1, 'dept_id', 'left_anti').show()                    # d04 IT
df2.groupBy('dept_id').count().filter(col('count') > 1).show()    # empty = unique
```

PREVIOUS ← Phase 3 · Intermediate Transformations
NEXT Phase 5 · Window Functions & UDFs →

Tip: keep all nine PDFs in the same folder so the previous / next links open the right file.